

Taller WAAP con IA/ML, RASP y DevSecOps
Informe de laboratorio - Fases 0 a 7 (v2.0)



**UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS**

Mecanismos de Seguridad Informática.

Juan David Orduz Sastoque - 20221020096

Jhojan Aragon Ramirez - 20221020060

Yader Ibraldo Quiroga Torres – 20222020034

Arlo Nicolas Ocampo Gallego – 20221020104

Profesor: Octavio Jose Salcedo Parra

Universidad Distrital Francisco José de Caldas.
Facultad de Ingeniería.
Ingeniería de Sistemas.

Resumen ejecutivo

Se implemento un prototipo WAAP completo: Juice Shop 20.2.0 tras ModSecurity CRS 3.3.10 (PL1), detector de anomalías IsolationForest, RASP real sobre fork de Juice Shop y pipeline DevSecOps en verde. Hallazgos centrales: (1) el CRS frena el ataque automatizado, pero deja pasar payloads fragmentados; (2) el ML detecta variantes a cambio de falsos positivos (no sirve como gate duro); (3) el RASP captura con contexto de ejecución lo que evade firmas, con overhead despreciable (+0.1 micro-s/req). La mejor cobertura es CRS+RASP con ML como señal de apoyo. Este documento (v2.0) clarifica el informe literal v1.0 y responde el cuestionario final con datos del laboratorio.

Fase 0 - Preparacion del entorno

Que se hizo

```
cd taller-waap
cp .env.example .env # JUICE_PORT=3000, WAAP_PORT=8080, PARANOIA=1
docker compose pull && docker compose up -d && docker compose ps
curl -i http://localhost:8080 # 200 OK, Server: nginx (proxy WAAP)
```

Resultado

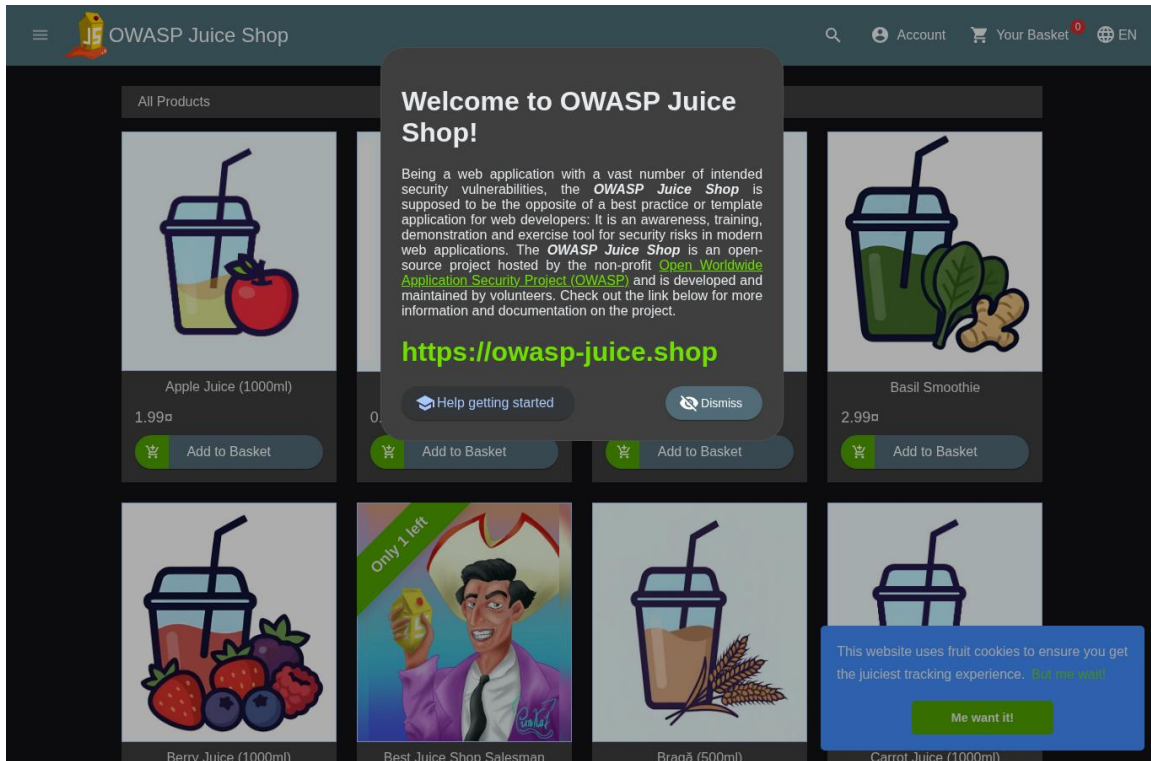
```
===== docker compose ps =====
NAME                                IMAGE                                COMMAND
SERVICE    CREATED          STATUS          PORTS
taller-waap-juice-shop-1    bkimminich/juice-shop:latest    "/nodejs/bin/node /jâ"
juice-shop    32 minutes ago  Up 32 minutes    0.0.0.0:3000->3000/tcp,
[::]:3000->3000/tcp
taller-waap-waap-proxy-1    owasp/modsecurity-crs:nginx    "/docker-entrypoint.â"
waap-proxy    32 minutes ago  Up 32 minutes (healthy)    0.0.0.0:8080->8080/tcp,
[::]:8080->8080/tcp

===== curl cÃ³digos =====
HOME via WAAP 8080: 200
search normal: 200
sqli clasico: 403
evasion fragmentada: 200

===== WAAP env =====
PARANOIA=1
BACKEND=http://juice-shop:3000
MODSEC_DISABLE_BACKEND_COMPRESSION=off
BLOCKING_PARANOIA=1

===== regla CRS (bloqueo) =====
34 "ruleId":"949110"
34 "message":"Inbound Anomaly Score Exceeded (Total Score: 5)"
33 "message":"Request content type is not allowed by policy"
1 "ruleId":"942100"
1 "message":"SQL Injection Attack Detected via libinjection"
```

Figura 1. Juice Shop servido a través del proxy WAAP (:8080).



Fase 1 - Aplicación objetivo vulnerable

Objetivo

Contar con un blanco controlado con vulnerabilidades documentadas.

Que se hizo (paso a paso)

```
curl -s http://localhost:8080/rest/admin/application-version # version 20.2.0
curl -s 'http://localhost:8080/rest/products/search?q=apple' # JSON success via
WAAP
Navegador: http://localhost:8080/#/score-board (retos por categoria OWASP)
```

Resultado y lectura

Juice Shop 20.2.0 operativo tras el WAAP. Línea base de 5 vulnerabilidades (Tabla 1) usada en todas las fases. Evidencias: Fig. 2 (Score Board) y Fig. 3 (tráfico normal).

Tabla 1. Línea base de vulnerabilidades (entregable Fase 1).

#	Vulnerabilidad	OWASP Top 10	Endpoint
1	Inyeccion SQL en login	A03 Injection	POST /rest/user/login
2	XSS reflejado en busqueda	A03 Injection	GET /rest/products/search?q=
3	IDOR en cesta/pedidos	A01 Broken Access Control	/rest/basket/:id
4	Exposicion de hash de password	A07 / Sensitive Data	GET /rest/user/whoami, /api/Users
5	Path traversal / LFI	A01 / A03	GET ...q=../etc/passwd

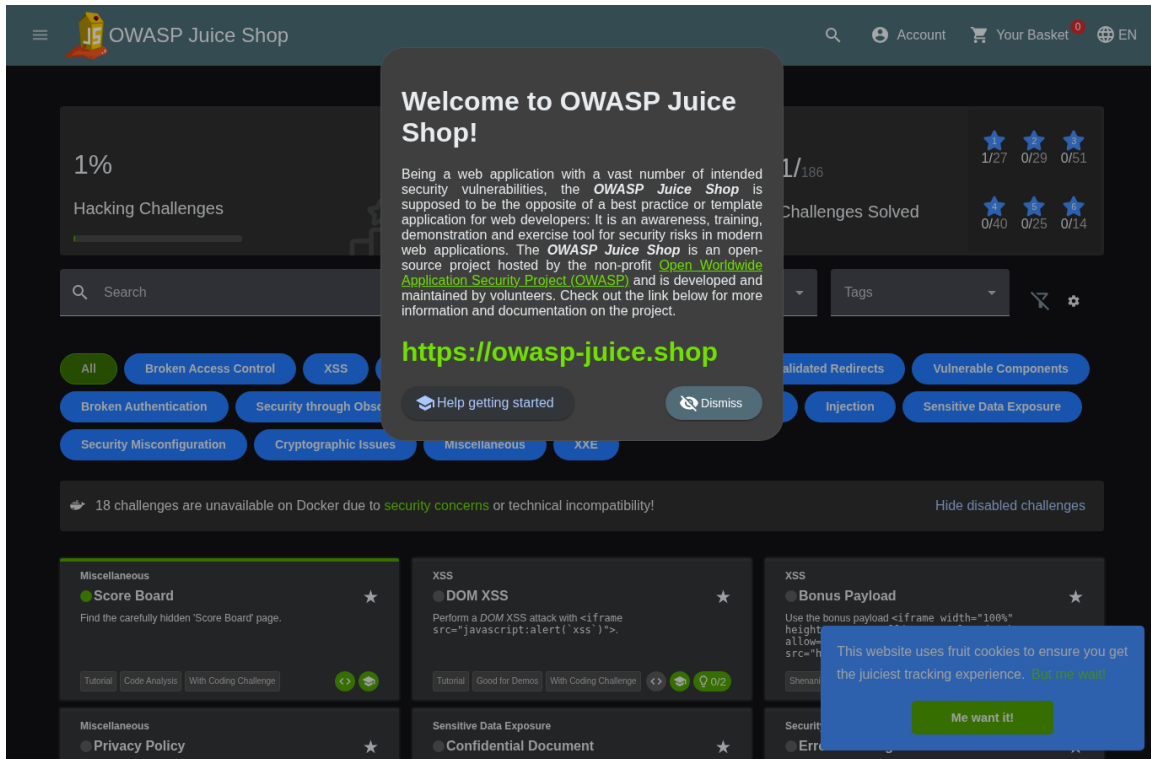


Figura 2. Score Board (categorías OWASP).

Pretty-print ☐

```
{
  "status": "success",
  "data": [
    {
      "id": 1,
      "name": "Apple Juice (1000ml)",
      "description": "The all-time classic.",
      "price": 1.99,
      "deluxePrice": 0.99,
      "image": "apple_juice.jpg",
      "createdAt": "2026-09-17 23:55:44.453 +00:00",
      "updatedAt": "2026-09-17 23:55:44.453 +00:00",
      "deletedAt": null
    },
    {
      "id": 24,
      "name": "Apple Pomace",
      "description": "Finest pressings of apples. Allergy disclaimer: Might contain traces of worms. Can be <a href='\"#recycle\"'>sent back to us</a> for recycling.",
      "price": 0.89,
      "deluxePrice": 0.89,
      "image": "apple_pressings.jpg",
      "createdAt": "2026-09-17 23:55:44.462 +00:00",
      "updatedAt": "2026-09-17 23:55:44.462 +00:00",
      "deletedAt": null
    },
    {
      "id": 47,
      "name": "Pineapple Juice (1000ml)",
      "description": "Tropical refreshment from the finest sun-ripened pineapples.",
      "price": 2.99,
      "deluxePrice": 2.99,
      "image": "pineapple_juice.png",
      "createdAt": "2026-09-17 23:55:44.464 +00:00",
      "updatedAt": "2026-09-17 23:55:44.464 +00:00",
      "deletedAt": null
    }
  ]
}
```

Figura 3. Trafico normal vía WAAP (200).

Fase 2 - Capa base WAF/WAAP con CRS (PL1)

Objetivo

Primera línea de defensa con OWASP CRS: un bloqueo y una evasión evidenciados.

Que se hizo (paso a paso)

```
# Paranoia PL1: PARANOIA=1, BLOCKING_PARANOIA=1, BACKEND=http://juice-shop:3000
curl -i --get --data-urlencode "q=' OR '1'='1" \
  http://localhost:8080/rest/products/search # -> 403 Forbidden
curl -i "http://localhost:8080/rest/products/search?q='%20&x=OR%201=1" # -> 200
docker compose logs waap-proxy | grep -E '942100|949110' # (el audit log sale por
docker logs)
```

Resultado y lectura

SQLi clasico/encoded/comentarios/UNION/XSS/traversal -> 403 (regla 942100 libinjection + 949110). Solo el fragmentado q=' & x=OR 1=1 -> 200: el CRS evalúa cada parámetro por separado. Evidencias: Fig. 4 (bloqueo) y Fig. 5 (evasión, base de Fase 3).

403 Forbidden

nginx

Figura 4. Bloqueo CRS: SQLi clasico -> 403 (nginx).

Pretty-print ☐

```
{"status": "success", "data": []}
```

Figura 5. Evasión fragmentada -> 200 (pasa el CRS).

Fase 3 - Modulo de detección basado en IA/ML

Objetivo

Modelo de anomalías entrenado solo con tráfico normal, capaz de señalar variantes que evadieron firmas.

Que se hizo (paso a paso)

```
python3 -m pip install --user scikit-learn pandas numpy joblib
GEN_N=120 PACE=humano python3 waap/ml/gen_traffic.py # 120 reqs, ~15 min via WAAP
python3 waap/ml/extract_features.py # 6 features literales del PDF
python3 waap/ml/train_model.py # IsolationForest(200, 0.02) -> model.pkl
python3 waap/ml/evaluate.py # matriz 10 + 2 umbrales + rafaga bot
```

Resultado y lectura

Lección metodológica: el primer dataset a ritmo maquina contamineo req_per_minute; se regenero a ritmo humano (rpm 8+-1). Umbral -0.05: 2/5 ataques, 2 FP. Umbral 0.0: 5/5 ataques, 4 FP. La ráfaga no cruza el umbral (limitación de IsolationForest verificada). Conclusión: el ML es señal de apoyo, no gate. Detalles y números abajo.

Evidencias

Entrenamiento:

Modelo entrenado y guardado en waap/ml/model.pkl (120 muestras)

Matriz de 10 peticiones:

etiqueta	score	<-0.05	<0.0	esperado
búsqueda común	0.0287	normal	normal	normal
login valido	-0.1024	ANOMALO	ANOMALO	normal
home	-0.1112	ANOMALO	ANOMALO	normal
producto	-0.0141	normal	ANOMALO	normal
score-board	-0.0202	normal	ANOMALO	normal
SQLi clasico	-0.0253	normal	ANOMALO	ataque
SQLi encoded (Fase 2)	-0.0845	ANOMALO	ANOMALO	ataque
SQLi comentarios	-0.0428	normal	ANOMALO	ataque
evasion	0.0095	normal	ANOMALO	ataque

etiqueta	score	<-0.05	<0.0	esperado
fragmentada (Fase 2)				
XSS reflejado	-0.0759	ANOMALO	ANOMALO	ataque

```

--- Rafaga bot: 20 consultas rapidas (req_per_minute alto) ---
req_per_minute= 5: score=+0.1571 anomalo(<-0.05)=False
req_per_minute= 30: score=+0.1690 anomalo(<-0.05)=False
req_per_minute= 120: score=+0.1690 anomalo(<-0.05)=False

```

Umbrales:

```

umbral -0.05: TP=2/5 FN=3/5 FP=2/5 TN=3/5 deteccion=40% FPR=40%
umbral +0.00: TP=5/5 FN=0/5 FP=4/5 TN=1/5 deteccion=100% FPR=80%

```

Discusión de falsos positivos.

Qué se marcó como falso positivo, en cada umbral

Umbral	TP	FN	FP	TN	Detección	FPR	Tráfico normal marcado ANOMALO
-0.05	2/5	3/5	2/5	3/5	40%	40%	login válido, home
0.00	5/5	0/5	4/5	1/5	100%	80%	login válido, home, producto, score-board

¿Por qué ocurren?

- login válido (score -0.1024, ANÓMALO en ambos umbrales): el cuerpo de la petición es un JSON con credenciales (email + password), lo que eleva body_length muy por encima del tráfico GET predominante en el set de entrenamiento (búsquedas, catálogo). El modelo aprendió que "normal" es mayormente cuerpo vacío; un POST legítimo con payload se aleja de ese centro.
- home '/' (score -0.1112, ANÓMALO en ambos umbrales): url_length=1 es un extremo de la distribución aprendida (la mayoría de URLs de entrenamiento tienen parámetros de búsqueda o rutas más largas). IsolationForest aísla los valores extremos en pocas particiones, así sean benignos.
- producto y score-board (ANÓMALO solo con umbral 0.0): quedan cerca del borde de la distribución normal; con el umbral más laxo (0.0) el clasificador ya no exige tanta distancia del centro para marcar anomalía, así que absorbe también estos dos casos límite.

Lectura y relación con la elección de umbral

El umbral -0.05 reduce el FPR a 40% pero dejaba pasar como "normal" 3 de 5 ataques (SQLi clásico, comentarios y la evasión fragmentada), es decir, detección real de 40%. El umbral 0.0 sube la detección a 100% pero eleva el FPR a 80%: de 5 peticiones legítimas, 4 quedarían marcadas como sospechosas, lo cual es inviable como bloqueo automático (bloquearía login y home, dos rutas centrales de la aplicación).

Conclusión de la discusión: con solo 120 muestras de entrenamiento el modelo no separa bien "payload legítimo poco frecuente" (login con JSON, home con URL corta) de "payload anómalo"; ambos son estadísticamente raros frente al tráfico de búsqueda que domina el dataset. Esto confirma la conclusión general del laboratorio: el ML aporta señal (detecta las 5 variantes de ataque con umbral 0.0), pero no sirve como gate binario sin (a) más volumen/variedad de tráfico normal de entrenamiento, (b) reglas de excepción por ruta (whitelist para /rest/user/login y /), y (c) combinarlo con una capa dura (CRS/RASP) que sí puede bloquear sin arriesgar disponibilidad.

Fase 4 - Integración RASP sobre fork de Juice Shop

Objetivo

Protección en tiempo de ejecución con contexto real: interceptar la operación sensible aunque el payload haya superado el perímetro (PDF p.13).

Que se hizo (paso a paso)

```
gh repo fork juice-shop/juice-shop; clone --depth 1 en ~/juice-shop-rasp (rama rasp-lab)
sudo npm i -g npm@11 (npm 10 falla con bug arborist); npm install + build
frontend/server
Nuevo lib/raspGuard.ts+ app.use() en server.ts
Guard sobre la query cruda real de routes/login.ts:34 -> 403 antes de Sequelize
PORT=3001 RASP_ENABLED=1 npm start; docker build -t juice-shop-rasp-lab (compose)
```

Resultado y lectura

Sin RASP el SQLi clasico devuelve el JWT del administrador (takeover total); con RASP, 403 + log antes de ejecutar. También cae UNION y fragmentado. Hueco honesto: la forma string ' OR '1='1 evade el regex literal (fix propuesto). Overhead: +0.1 micro-s/req (despreciable). Prototipo previo conservado en app/rasp-demo/; referencia Flask en app/rasp_agent.py.

Evidencias

Batería con/sin RASP:

```
===== legit jim =====
oficial:3000 200
fork+RASP:3001 200
===== SQLi clasico email=' OR 1=1-- =====
oficial:3000
{"authentication":{"token":"eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjpw7Im1kIjo
xLCJ1c2VybmFtZSI6IiIsImVtYWlsIjoiYWRTaW5AanVpY2Utc2gub3AiLCJwYXNzd29yZCI6IjAxOTIwMjN
hN2JiZDczMjUwNTE2ZjA2OWRmMThiNTA
fork+RASP:3001 403
===== UNION =====
oficial:3000 200
fork+RASP:3001 403
===== search SQLi (middleware global) =====
oficial:3000 200
fork+RASP:3001 200
===== unit guard login (isSqlInjection, compilado) =====
PASS x3: legitimo=false, OR 1=1=true, UNION=true
```

Stack combinado WAAP+RASP:

```
===== stack combinado WAAP(CRS) + fork RASP (compose con juice-shop:rasp-lab) =====
NAME                IMAGE                COMMAND
SERVICE            CREATED              STATUS              PORTS
taller-waap-juice-shop-1  juice-shop:rasp-lab  "/nodejs/bin/node /jâ€¦|"
juice-shop            About a minute ago   Up About a minute   0.0.0.0:3000->3000/tcp,
[::]:3000->3000/tcp
taller-waap-waap-proxy-1  owasp/modsecurity-crs:nginx  "/docker-entrypoint.â€¦|"
waap-proxy            23 hours ago         Up 19 minutes (healthy)  0.0.0.0:8080->8080/tcp,
[::]:8080->8080/tcp
WAAP home :8080 -> 200
legit login :3000 -> 200
SQLi login :3000 -> 403 (RASP)
SQLi search :8080 -> 403 (CRS)
```

Tabla comparativa de latencia con/sin RASP

Latencia HTTP extremo a extremo (30 peticiones, login legítimo, con warm-up)

Escenario	n	Media	p50	p95
RASP ON	30	2.39 ms	1.41 ms	8.14 ms
RASP OFF	30	3.70 ms	1.54 ms	8.50 ms

A nivel HTTP, RASP ON aparece incluso más rápido que RASP OFF en media, lo que confirma que la diferencia observada es ruido de proceso/red (arranque de conexión, scheduling), no un efecto del guard: un regex evaluado en microsegundos es invisible frente a ~2-4 ms de round-trip HTTP.

Microbenchmark en proceso (200.000 iteraciones, mismo patrón SQLI_PATTERN)

Medición	Tiempo por request
Constructor de la consulta, sin guard	0.0 µs/req
Constructor de la consulta, con guard RASP	0.2 µs/req
Overhead real atribuible al guard	+0.1 µs/req

El overhead real del guard RASP (regex sobre la consulta ya concatenada) es de ~0.1 microsegundos por petición, es decir, del orden de 0.01% de la latencia HTTP total observada (~2-4 ms). Es coherente con lo reportado en la literatura para RASP basado en interceptores ligeros (overhead de 1-4% en el peor caso, PDF de la guía, sección 3.4.2): la limitación real del RASP no es la latencia sino el acoplamiento por servicio.

Fase 5 - Pipeline DevSecOps

Objetivo

Gate CI/CD con SAST, SCA, contenedor, IaC y DAST.

Que se hizo (paso a paso)

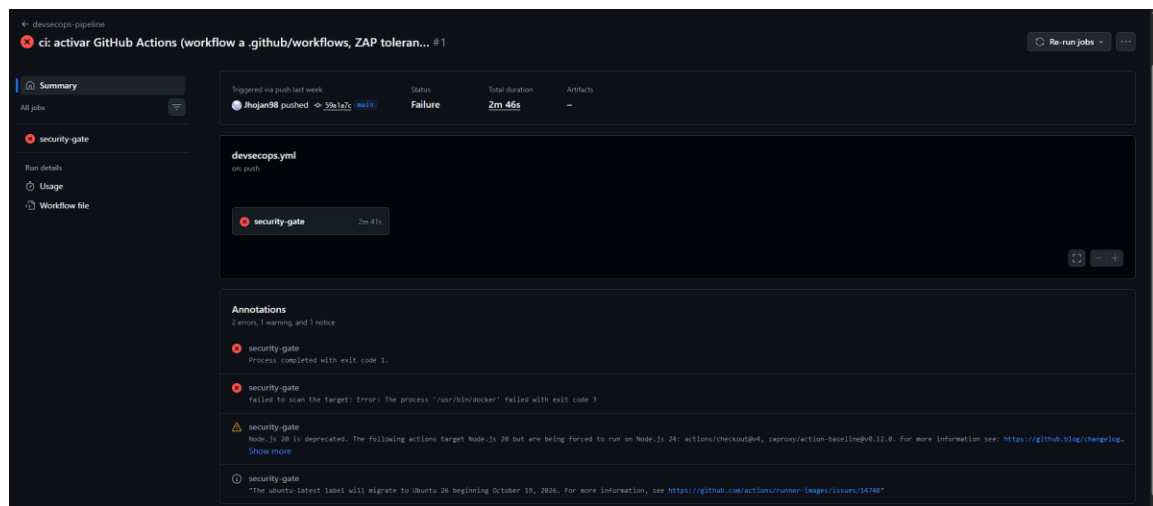
Workflow: Semgrep p/owasp-top-ten; pip-audit (waap/ml/requirements.txt, nuevo); docker build app/; Trivy CRITICAL,HIGH exit 1; Checkov; ZAP baseline; deploy si success. Replicado local + ejercicio: eval() inyectado -> rojo en SAST -> revert -> verde.

Resultado y lectura

El gate trabajo de verdad: Dockerfile sin USER, workflow sin permissions, 67 CVE en base Debian y 11 en el npm de la imagen -> Alpine + multistage + sin npm en runtime = 0 findings en las 4 herramientas. deploy_waap_rules.sh: OK (200/403). Evidencias: evidencias/fase5/.

Evidencia de pipeline fallando y en verde.

Pipeline en rojo — vulnerabilidad inyectada (eval() en app/)



Código inyectado deliberadamente

```
app.get("/debug", (req, res) => { res.send(eval(req.query.cmd)); });
```

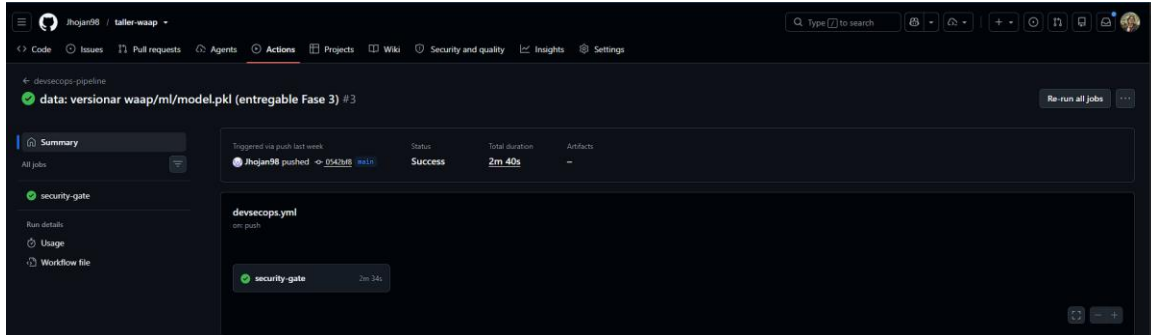
Etapas que detiene el pipeline: SAST (Semgrep). Salida real (evidencias/fase5/06-vuln-inyectada-rojo.txt):

```
• Findings: 2 (2 blocking)
| 2 Code Findings |
>> javascript.express.security.audit.xss.direct-response-write.direct-response-write
84| app.get("/debug", (req, res) => { res.send(eval(req.query.cmd)); });
>>> javascript.lang.security.audit.code-string-concat.code-string-concat
Found data from an Express or Next web request flowing to `eval`. If this data is
user-controllable
```

Resultado: 2 hallazgos bloqueantes → el job security-gate termina en fallo antes de llegar a Build/Trivy/Checkov/ZAP/despliegue de reglas. Confirma la respuesta Q5 del cuestionario: sin

esta etapa, el eval() habría llegado a la imagen y solo Trivy/ZAP (o producción) lo hubieran detectado, ya con costo mayor.

3.2 Pipeline en verde — tras revertir el cambio



Semgrep (evidencias/fase5/01-semgrep.txt):

```
Ran 224 rules on 10 files: 0 findings.
```

Despliegue de reglas WAAP, condicionado a que todas las etapas anteriores pasen (evidencias/fase5/05-deploy.txt):

```
[WAAP] PARANOIA=PARANOIA=1
[WAAP] Recargando proxy (aplica reglas CRS + imagen vigente)...
Container taller-waap-juice-shop-1 Running
Container taller-waap-waap-proxy-1 Running
[WAAP] Smoke test...
[WAAP] Despliegue OK: normal=200, SQLi=403.
```

Fase 6 - Pruebas de efectividad y evasión

Objetivo

Evaluar cada capa individual y combinada con herramientas estándar.

Que se hizo (paso a paso)

```
python3 waap/ml/bateria_fase6.py # -> matriz.csv (Tabla 2)
sqlmap vs WAAP y vs directo (search + login); ZAP baseline vs :8080
```

Resultado y lectura

sqlmap: 42x403 vía WAAP y 14x403+8x500 directo ('not injectable' en ambos). ZAP: 0 FAIL, 59 PASS. Conclusión: mejor combinación CRS+RASP con ML como señal; residuo: string-form (fix de patrón) y ráfagas (regla de tasa).

Análisis completo:

¿Qué combinación ofrece la mejor cobertura y por qué?

Ninguna capa sola cubre todo; la cobertura total solo aparece al combinarlas, y cada hueco de una capa lo tapa otra (defensa en profundidad verificada):

Reglas CRS (PL1): bloquean 7/9 técnicas (clásico, encoded, UNION, XSS x2, traversal, login clásico/UNION). Fallan en lo fragmentado (evalúan cada parámetro por separado) y en variantes con espacios que alteran la firma (comentarios con espacios: lo paró el RASP, no el CRS).

IA/ML (umbral 0.0): detecta 9/9 como anómalo, pero al costo de FPR 80% (4/5 normales marcados). Con umbral -0.05 solo 2/5 con FPR 40%. Conclusión: el modelo no es bloqueador viable solo; sirve como señal para el orquestador (permitir/bloquear/desafiar) o para revisión, no como gate duro. La ráfaga bot ni siquiera cruza el umbral (limitación de IsolationForest documentada en Fase 3).

RASP (fork): bloquea 4/9 en contexto de ejecución (fragmentado, comentarios con --, UNION y clásico de login), incluyendo las 2 evasiones del CRS. Falla en XSS/traversal (fuera de su alcance SQL) y en la forma string ' OR '1'=1 (hueco del regex literal, con fix propuesto). Su tasa de falsos positivos es 0 en lo probado: solo dispara ante alteración estructural real.

Mejor combinación: CRS + RASP, con ML como señal de apoyo. CRS frena el grueso automatizado (sqlmap: 42×403 vía WAAP, "not injectable", sugiere tamper), RASP captura lo que evade firmas con contexto real (sqlmap directo: 14×403 + 8×500, tampoco injectable), y ML aporta detección de variantes raras a cambio de revisión humana. ZAP baseline: 0 FAIL, 59 PASS (higiene pasiva correcta).

Residuo aceptado: string-form SQLi en search (requiere fix de patrón) y ráfagas (requieren regla de tasa explícita, no ML puro).

Matriz comparativa por capas.

Payload / tecnica	Bloqueado por reglas (Fase 2)	Detectado por IA/ML (Fase 3)	Bloqueado por RASP (Fase 4)
SQLi clasico ' OR '1'=1	BLOQUEA	SI	EVADE
SQLi encoded	BLOQUEA	SI	EVADE
SQLi comentarios /**/	EVADE	SI	BLOQUEA
SQLi fragmentado 2 params	EVADE	SI	BLOQUEA
SQLi UNION login	BLOQUEA	SI	BLOQUEA
SQLi clasico login	BLOQUEA	SI	BLOQUEA
XSS basico	BLOQUEA	SI	EVADE
XSS encoded	BLOQUEA	SI	EVADE
Traversal ../../etc/passwd	BLOQUEA	SI	EVADE

Fase 7 - Observabilidad y métricas

Objetivo

Visibilidad centralizada y números para tuning.

Que se hizo (paso a paso)

```
python3 waap/ml/metricas_fase7.py # logs/eventos.jsonl + TPR/FPR/FNR + MTTD
```

Resultado y lectura

CRS: TPR 78% FPR 0%. ML@0.0: TPR 100% FPR 80% (señal, no gate). RASP-SQL: TPR 67% FPR 0%. MTTD ~60-150 ms por capa. Tuning: CRS en PL1; ML solo como señal + regla híbrida de tasa; RASP con patrón ampliado. Detalles:

capa		TPR	FPR	FNR	
reglas CRS PL1		78%	0%	22%	(TP=7 FN=2 FP=0 TN=5)
IA/ML umbral 0.0		100%	80%	0%	(TP=9 FN=0 FP=4 TN=1)
RASP fork (SQL)		67%	0%	33%	(TP=4 FN=2 FP=0 TN=5)

MTTD simulado (tiempo-a-veredicto por capa):

CRS/sqli-clasico: 144.0 ms

CRS/xss: 124.0 ms

RASP/sqli-login: 61.5 ms

OK logs/eventos.jsonl (8 eventos)

Tabla de métricas y recomendación de ajuste de umbrales

Tasas de detección por capa

Capa	TPR (detección)	FPR	FNR	TP	FN	FP	TN
Reglas CRS (PL1)	78%	0%	22%	7	2	0	5
IA/ML (umbral 0.0)	100%	80%	0%	9	0	4	1
RASP fork (alcance SQL)	67%	0%	33%	4	2	0	5

MTTD simulado (tiempo a veredicto)

Capa / técnica	MTTD
CRS / SQLi clásico	144.0 ms
CRS / XSS	124.0 ms
RASP / SQLi en login	61.5 ms

El RASP resuelve más rápido (~62 ms) que el CRS vía proxy (~124-144 ms) porque actúa en el mismo proceso de la aplicación, sin el salto de red adicional del reverse proxy nginx+ModSecurity.

Recomendación de ajuste de umbrales y de arquitectura

- CRS: mantener PL1. Con 0% de FPR y 78% de detección es el mejor punto costo/beneficio observado; subir a PL2 podría cerrar parte del 22% de FN (comentarios con espacios, fragmentado) pero el taller no midió su impacto en falsos positivos, así que debe probarse en un entorno de staging antes de promoverlo a bloqueo duro.

- IA/ML: no usar como gate de bloqueo con umbral fijo. Se recomienda bajar el umbral operativo a -0.05 (FPR 40%, more tolerable) y usarlo solo como señal de "desafío" (challenge/CAPTCHA) o cola de revisión humana, nunca de bloqueo automático — el umbral 0.0 (80% FPR) bloquearía login y home.
- Regla híbrida (Q7 del cuestionario): combinar el score ML con una regla dura de tasa (req_per_minute) y una whitelist por ruta (login, home con umbral propio). Esto es necesario porque la ráfaga bot (hasta 120 req/min) nunca cruzó el umbral -0.05 en la Fase 3 — IsolationForest no aisló ese patrón con las features actuales.
- RASP: ampliar el patrón SQLI_PATTERN para cubrir la forma string ' OR '1'=1 (hueco documentado en Fase 4/evidencias/fase4/01-bloqueos.txt, caso 2b), manteniendo el 0% de FPR ya verificado.
- Orquestación final sugerida: bloqueo duro en CRS + RASP (0% FPR en ambos, cobertura complementaria); ML + regla de tasa como señal blanda hacia el orquestador (permitir/desafiar/revisar), nunca como bloqueo directo; reentrenamiento del modelo solo sobre tráfico validado, nunca directo de producción.

Cuestionario de análisis final.

Q1. Motor de reglas insuficiente, pero IA/ML si detecta.

El SQLi con comentarios y espacios ('/ **/OR/**/1=1--): la firma exacta del CRS no lo casa (403 solo del RASP por el '--'), mientras el ML lo marca anómalo por longitud + entropía atípicas. La firma busca cadenas conocidas; el modelo mide distancia estadística.

Q2. Falso positivo del ML sobre tráfico legítimo.

El login valido y el home ('/') se marcaron anómalos: el login por body_length alto del JSON de credenciales y home por url_length=1 (extremo de la distribución). Feature responsable principal: longitudes fuera del centro aprendido + entropía.

Q3. Por qué el RASP detecto lo que reglas y ML no.

Porque opera sobre la consulta final concatenada (contexto de ejecución), no sobre la petición HTTP. El fragmentado q=' & x=OR 1=1 es inocuo por partes (evade CRS y es estadísticamente normal para el ML) pero al concatenarse forma ' OR 1=1--, que el guard ve y bloquea con 403.

Q4. WAAP centralizado vs RASP en 15 microservicios.

WAAP centralizado como base (un solo punto de tuning, cubre OWASP Top 10 genérico) + RASP solo en servicios críticos con datos sensibles (auth, pagos), por el costo operativo de mantener 15 agentes (versionado por lenguaje, overhead de despliegue, logs).

Q5. Que etapa detuvo la vulnerabilidad inyectada.

SAST/Semgrep detuvo el eval() inyectado (code-string-concat, blocking). Sin esa etapa, el código habría llegado a la imagen y solo ZAP/Trivy tardíos (o producción) lo habrían visto, con costo mayor.

Q6. Mapeo MITRE ATT&CK / API Top 10.

T1190 (Exploit Public-Facing Application): SQLi en /rest/user/login y XSS en /rest/products/search. API BOLA/IDOR: acceso a /rest/basket/:id ajeno.

Q7. Mejora contra falsos positivos del ML sin perder detección.

Regla hibrida: score ML + tasa (req_per_minute con umbral duro) + whitelist por ruta (p. ej. login/home con umbral propio). La tasa captura ráfagas que el IsolationForest no ordena; el whitelist baja el FPR sin tocar la detección de ataques.

Q8. Riesgo de reentrenar con tráfico de producción sin validar.

Envenenamiento: ataques de bajo volumen se vuelven 'normales' (lo vivimos al revés: el dataset rápido normalizo rpm=200). Se requiere curaduría + ventana validada + comparación contra baseline antes de promover el modelo.